

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Principios de Sistemas Operativos - CE4303

Laboratorio 1

My Name Booteable

Profesor: Jason Leiton Jimenez

Alumnos:

Mauro Andrés Navarro Obando

Mauricio Luna Acuña

Grupo: 1

Marzo 2026

Índice

1. Estructura del proyecto	2
1.1. Descripción rápida de cada archivo	2
2. Implementación Legacy	2
2.1. Idea general	2
2.2. Primera etapa: <code>boot.asm</code>	3
2.3. Segunda etapa: <code>myname.asm</code>	3
2.4. Interrupciones BIOS utilizadas	3
2.5. Flujo de ejecución Legacy	4
3. Implementación UEFI	4
3.1. Idea general	4
3.2. Aleatoriedad en UEFI	4
3.3. Flujo de ejecución UEFI	4
4. Automatización con Makefile	5
4.1. Comandos principales	5
5. Herramientas utilizadas	5
6. Resultados obtenidos	5

1. Estructura del proyecto

La estructura general utilizada fue la siguiente:

```
src/  
  boot.asm  
  myname.asm  
  uefi/  
    main.c  
  
build/  
uefi_disk/  
  EFI/  
    BOOT/  
      BOOTX64.EFI  
  
Makefile
```

1.1. Descripción rápida de cada archivo

Archivo	Descripción
boot.asm	Primera etapa del arranque Legacy. Muestra la pantalla inicial, espera ENTER y carga la segunda etapa desde disco.
myname.asm	Segunda etapa del arranque Legacy. Implementa la lógica principal del programa.
main.c	Implementación UEFI en C. Ejecuta la versión moderna del mismo comportamiento.
Makefile	Automatiza la compilación y ejecución tanto de la versión Legacy como de la versión UEFI.

2. Implementación Legacy

2.1. Idea general

La versión Legacy fue desarrollada en ensamblador x86 de 16 bits, trabajando en modo real. Debido a la limitación clásica del boot sector de 512bytes, se decidió dividir la implementación en dos etapas:

- **Stage 1:** boot.asm
- **Stage 2:** myname.asm

Esta separación permitió mantener la primera etapa pequeña y enfocada únicamente en el arranque, mientras que la segunda etapa contiene la lógica del juego.

2.2. Primera etapa: `boot.asm`

La primera etapa realiza las siguientes acciones:

1. Inicializa los registros principales.
2. Configura una pila simple.
3. Guarda la unidad de arranque.
4. Limpia la pantalla.
5. Muestra un mensaje inicial.
6. Espera a que el usuario presione `ENTER`.
7. Carga la segunda etapa desde el dispositivo de arranque.
8. Transfiere el control a la dirección `0000:8000`.

2.3. Segunda etapa: `myname.asm`

La segunda etapa contiene la lógica principal del programa Legacy:

- cálculo de posición inicial pseudoaleatoria,
- dibujo del nombre en pantalla,
- cambio de orientación según las flechas,
- reinicio de posición con `R`,
- reinicio de la máquina con `ESC`.

2.4. Interrupciones BIOS utilizadas

Interrupción	Uso	Descripción
<code>int 10h</code>	Video	Limpieza de pantalla, posicionamiento de cursor e impresión de caracteres.
<code>int 16h</code>	Teclado	Espera y lectura de teclas.
<code>int 13h</code>	Disco	Lectura de sectores para cargar la segunda etapa.
<code>int 1Ah</code>	Tiempo	Obtención de ticks del reloj BIOS para generar pseudoaleatoriedad.
<code>int 19h</code>	Reinicio	Reinicio simple del arranque.

Tabla 2: Interrupciones utilizadas en la versión Legacy

2.5. Flujo de ejecución Legacy

BIOS carga el sector de arranque → se ejecuta <code>boot.asm</code> → se muestra bienvenida → usuario presiona ENTER → se carga <code>myname.asm</code> → se ejecuta el programa principal
--

Figura 1: Flujo general de la versión Legacy

3. Implementación UEFI

3.1. Idea general

La implementación UEFI se desarrolló en C usando GNU-EFI. En este caso, el arranque ya no depende de un sector booteable de 512bytes, sino de una aplicación UEFI estándar ubicada en la ruta:

EFI/BOOT/BOOTX64.EFI

3.2. Aleatoriedad en UEFI

Para evitar que la posición inicial fuera siempre la misma, se utilizó una semilla mezclando:

- el `timestamp counter` del CPU (`rdtsc`),
- información de hora obtenida desde UEFI (`GetTime`),
- y una actualización interna de la semilla.

3.3. Flujo de ejecución UEFI

Firmware UEFI → busca <code>BOOTX64.EFI</code> → ejecuta <code>main.c</code> compilado como aplicación EFI → muestra bienvenida → usuario presiona ENTER → entra al ciclo principal del programa
--

Figura 2: Flujo general de la versión UEFI

4. Automatización con Makefile

4.1. Comandos principales

Comando	Función
<code>make legacy-build</code>	Compila la versión Legacy y genera <code>boot.img</code> .
<code>make legacy-run</code>	Compila y ejecuta la versión Legacy en QEMU.
<code>make uefi-build</code>	Compila la versión UEFI y genera <code>BOOTX64.EFI</code> .
<code>make uefi-run</code>	Compila y ejecuta la versión UEFI en QEMU con OVMF.
<code>make clean</code>	Limpia archivos generados.

Tabla 3: Comandos principales del Makefile

5. Herramientas utilizadas

- **Ubuntu:** entorno principal de desarrollo.
- **NASM:** ensamblador para la versión Legacy.
- **QEMU:** ejecución y prueba de las versiones simuladas.
- **OVMF:** firmware UEFI para ejecutar la versión UEFI en QEMU.
- **GNU-EFI:** librerías y archivos necesarios para construir `BOOTX64.EFI`.
- **GCC:** compilación de la versión UEFI en C.
- **Make:** automatización de compilación y ejecución.
- **VS Code:** edición del código fuente.

6. Resultados obtenidos

- **Legacy:** funcional en QEMU mediante una imagen booteable de dos etapas.

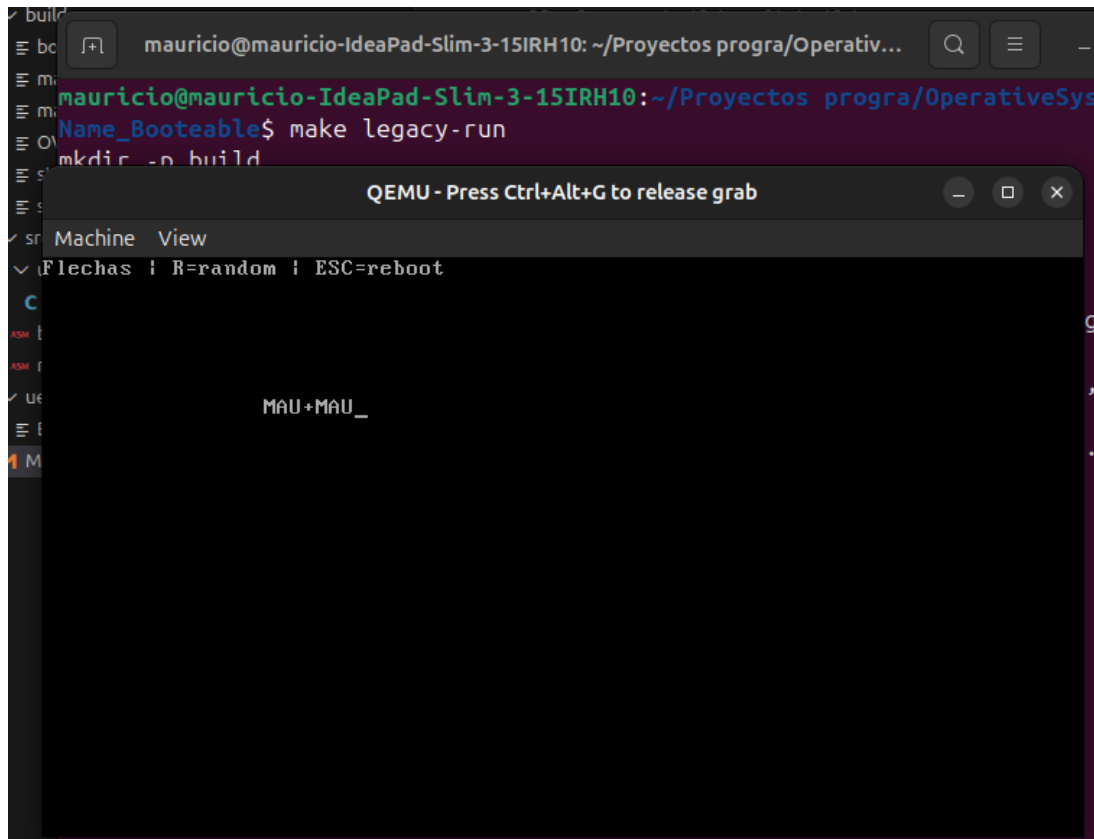


Figura 3: Ejecución de la versión Legacy en QEMU

- **UEFI**: funcional en QEMU y ejecutable desde USB en una máquina real.

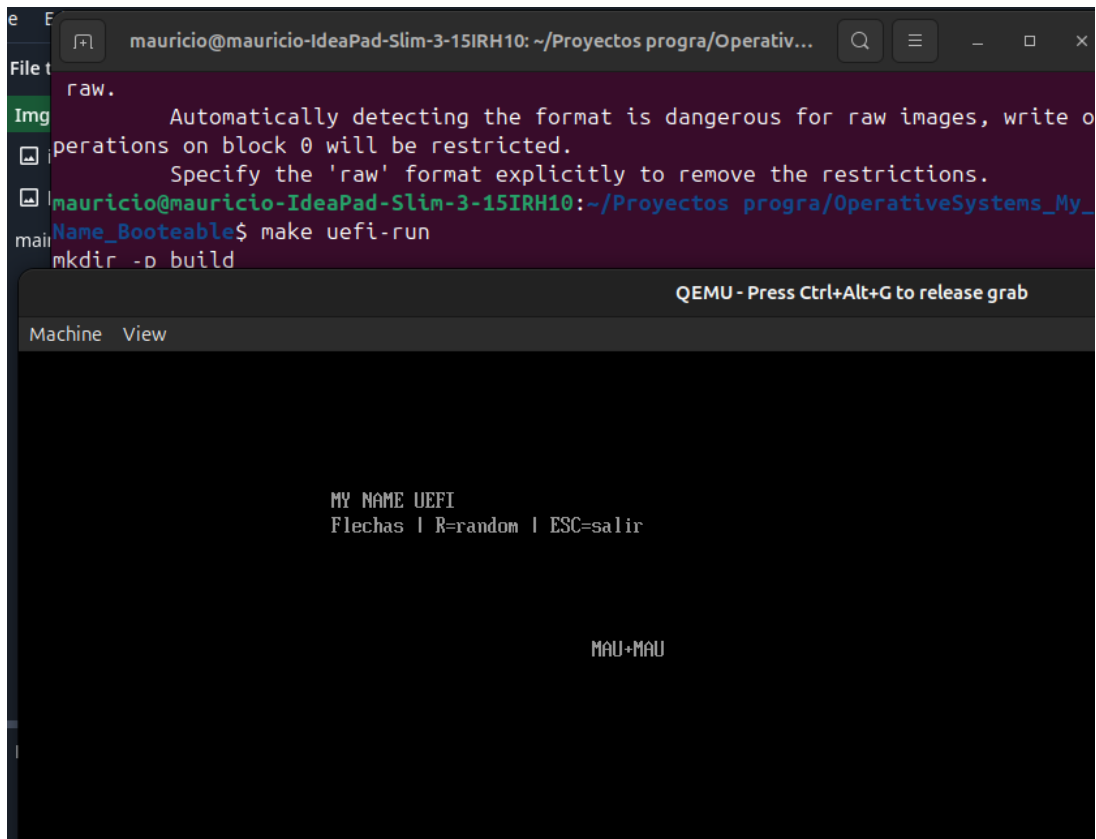


Figura 4: Ejecución de la versión UEFI en QEMU

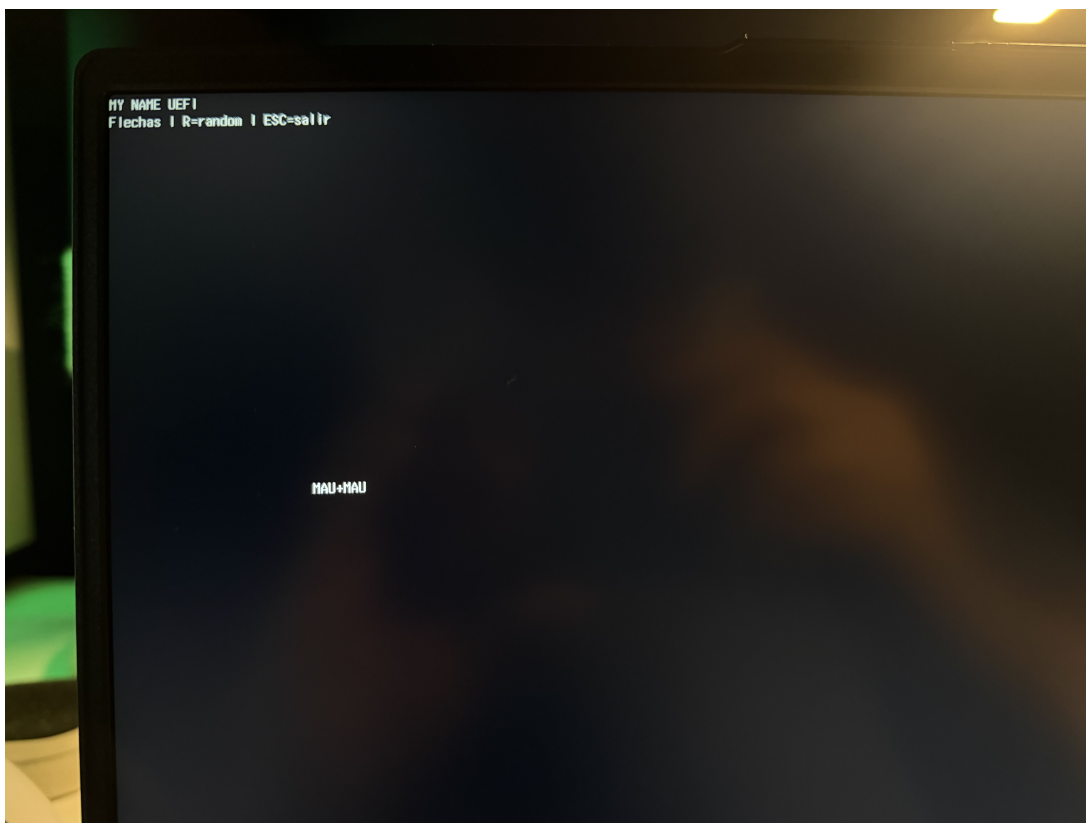


Figura 5: Ejecución de la versión UEFI en hardware real